

PROVA DI FINE

MODULO 2



MASTER DATA ANALYSIS AND AI

Merzetti Greta

QUESTIONS SECTION

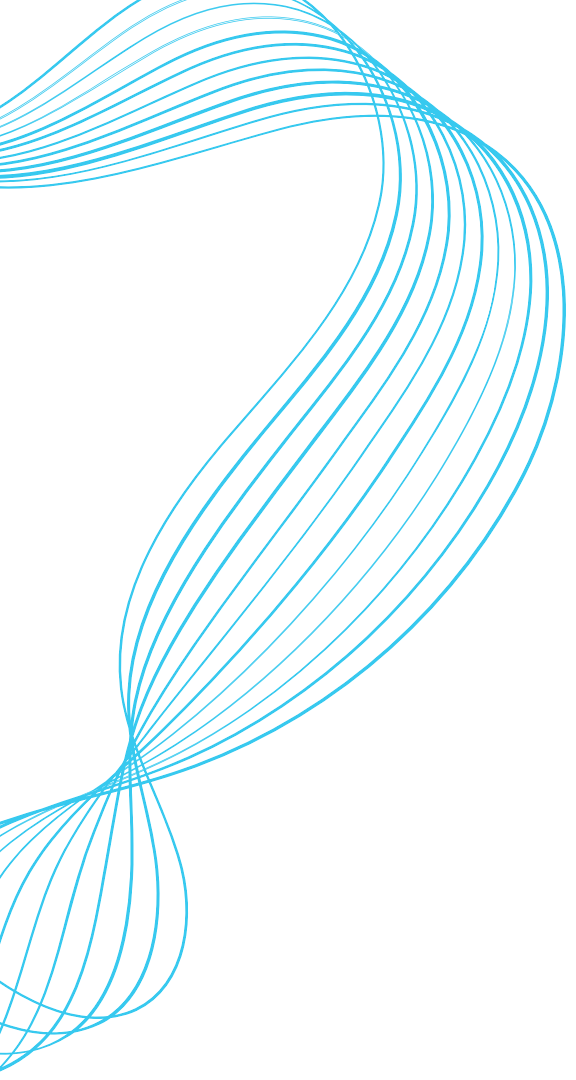
#cosa si intende per database?

Con database si intende una **collezione organizzata, strutturata e digitale** di dati in modo logico e coerente per facilitare operazioni di CRUD, creazione, lettura, aggiornamento e eliminazione di dati.

#cos'è un DBMS?

Un **DBMS** (Database Management system) è un software che consente la **creazione, la manipolazione e l'interrogazione** di database.





*/*Indica le principali clausole di uno statement SELECT in ordine di esecuzione logica. Descrivi per ciascuna delle clausole indicate la logica di funzionamento*/*

L'ordine di esecuzione logica non corrisponde all'ordine di scrittura per il DBMS.

Per quanto concerne le principali clausole di uno statement select queste sono: **from, join, where, group by, having e order by.**

In ordine di esecuzione le clausole vengono eseguite come descritto dal seguente elenco:

1. **From**, eseguita per **prima**, consente al **software** di comprendere la **tabella** da cui attingere per interrogare i record. Simultaneamente viene eseguita dal software la clausola **join**, grazie alla quale record provenienti da diverse tabelle vengono unite in un unico set di dati.
2. **Where**, eseguita per **seconda**, consente di **filtrare i record** in base a diverse possibili condizioni
3. **Group by**, eseguita per **terza**, consente di **raggruppare i record** in base a uno o più criteri
4. **Having**, eseguita per **quarta**, consente di filtrare i gruppi restituiti dalla group by
5. **Select**, eseguita per **quinta**, consente di definire i **campi** che si vogliono vedere restituiti nel **result set**
6. **Order by**, eseguita per **sesta**, consente di **ordinare i record**

/ Descrivi, immaginando uno scenario a te familiare, il concetto di group by. Utilizza l'approccio che ritieni più efficiente per trasmettere il concetto (suggerimento: disegna anche una sola tabella in Excel o in word con poche colonne e pochi record e descrivi, basandosi sulla tabella stessa, un esempio di group by). */*

LibroID	Titolo	Autore	AnnoUscita	Prezzo
1	Il nome della rosa	Umberto Eco	1980	14,00
2	Il pendolo di Foucault	Umberto Eco	1988	16,50
3	Baudolino	Umberto Eco	2000	12,00
4	Norwegian Wood	Haruki Murakami	1987	13,00
5	Kafka sulla spiaggia	Haruki Murakami	2002	14,00
6	1Q84	Haruki Murakami	2009	20,00
7	Cecità	José Saramago	1995	11,00
8	Le intermittenze	José Saramago	2005	10,50
9	La variante di Lüneburg	Paolo Maurensig	1993	9,00
10	Il sistema periodico	Primo Levi	1975	12,50

In questo caso group by potrebbe essere applicato all'autore del libro: potrei voler sapere la quantità di libri prodotti per autore con un prezzo medio maggiore di 12.

```
SELECT Autore,
COUNT(LibroID) AS NumeroLibri,
AVG(Prezzo) AS PrezzoMedio
FROM Libri
GROUP BY Autore
HAVING PrezzoMedio > 12
ORDER BY PrezzoMedio DESC;
```

MYSQL procederà a darmi un result set composto dall'autore, dalla funzione di aggregazione count(titolo), che mi restituirà il numero libri e dall'altra funzione di aggregazione avg del prezzo, che mi risestituirà la media dei prezzi all'interno dei gruppi; attraverso il group by raggrupperà appunto i record per autore, mentre attraverso il filtro per i gruppi having mi restituirà solo i gruppi di libri che hanno un prezzoMedio maggiore di 12.



#Descrivi la differenza tra uno schema OLTP e uno schema OLAP.

Lo schema **OLTP** (Online transactional processing) ha lo scopo di garantire **consistenza, integrità e sicurezza** delle transazioni, le quali vengono costantemente generate attraverso operazioni CRUD dagli utenti. Diversamente, il sistema **OLAP** (online analytical processing) consente **l'analisi e l'interrogazione** complessa di volumi di dati collettati nel data warehouse.

#Dato un medesimo scenario di analisi, qual è la differenza in termini di risultato ottenibile tra una join e una subquery?

La **Join** prende due tabelle e le incolla lateralmente in base a un elemento comune, si ottiene una **tabella combinata tra la tabella 1 e la tabella 2 in base alle diverse esigenze**.

La **Subquery** invece funge da domanda dentro un'altra domanda. Viene usata solitamente per **restringere il campo di ricerca della query principale, ottenendo solo i dati della tabella principale**. Non si possono vedere "direttamente" le colonne della tabella usata nella subquery.

#Cosa si intende per DML e DDL?

Con **DML**, data manipulation language, si intendono tutte quelle operazioni che concernono la **manipolazione dei dati**, quindi tutte le istruzioni che servono per modificare e interrogare i dati nelle tabelle. Con **DDL**, data definition language, si intendono invece tutte quelle operazioni che concernono istruzioni utili a **definire** – creare, modificare, eliminare-**tabelle**.

#Quali istruzioni possono utilizzare per estrarre l'anno da un campo data? Proponi degli esempi.

Da un campo data si può utilizzare l'istruzione **YEAR** per ottenere l'anno corrispondente alla data. Con la seguente query

```
SELECT OrdineID, YEAR(DataOrdine) AS AnnoVendita  
FROM Ordini;
```

**/ottengo quindi dalla tabella ordini due colonne corrispondenti all'ordine ID e l'anno in cui è stato effettuato l'ordine (estratto dal campo DataOrdine)*/*

#Qual è la differenza tra gli operatori logici AND e OR?

La differenza principale è che mentre **AND** restituisce **true** se **entrambe** le espressioni sono **true**, **OR** restituisce **true** se **una delle due** espressioni booleane è **true**.

Nel contesto di MySQL dunque, con **AND** affinché un record venga incluso nel result set **entrambe le condizioni devono risultare verificate** contemporaneamente. Diversamente, con **OR** il record viene incluso nel result set se **una delle due condizioni è verificata**.

#È possibile innestare una query nella clausola SELECT?

Si, è possibile attraverso una **subquery scalare** che quindi restituisce un valore unico e non un elenco che viene utilizzato come campo nel result set in tale caso.

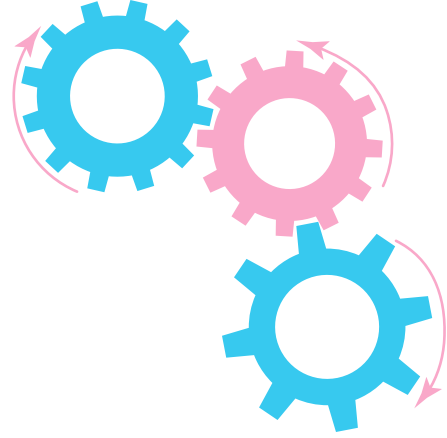
#Qual è la differenza tra l'operatore logico OR e l'operatore logico IN?

In tal caso la differenza è che **OR** restituisce **true** se **una delle due** espressioni risulta **true**, mentre **IN** restituisce **true** se l'operando è uguale a **una delle espressioni elencate**.

Nel contesto MySQL l'utilizzo di **IN** consente di ricercare la **corrispondenza in una lista di valori specifici**, mentre l'utilizzo di **OR** restituisce **tutti i record dove si verifica o una o l'altra condizione** (senza alcuna relazione tra le due).

#L'operatore logico BETWEEN include anche gli estremi del range specificato?

Sì, l'operatore **BETWEEN**, che consente di **verificare se un dato record è compreso nell'intervallo specificato dal between**, comprende **gli estremi del range specificato**.



TASKS MYSQL

1

Proponi una progettazione concettuale e logica della base dati.

2

Descrivi la struttura delle tabelle che reputi utili e sufficienti a modellare lo scenario proposto tramite la sintassi DDL. Implementa fisicamente le tabelle utilizzando il DBMS SQL Server(o altro).

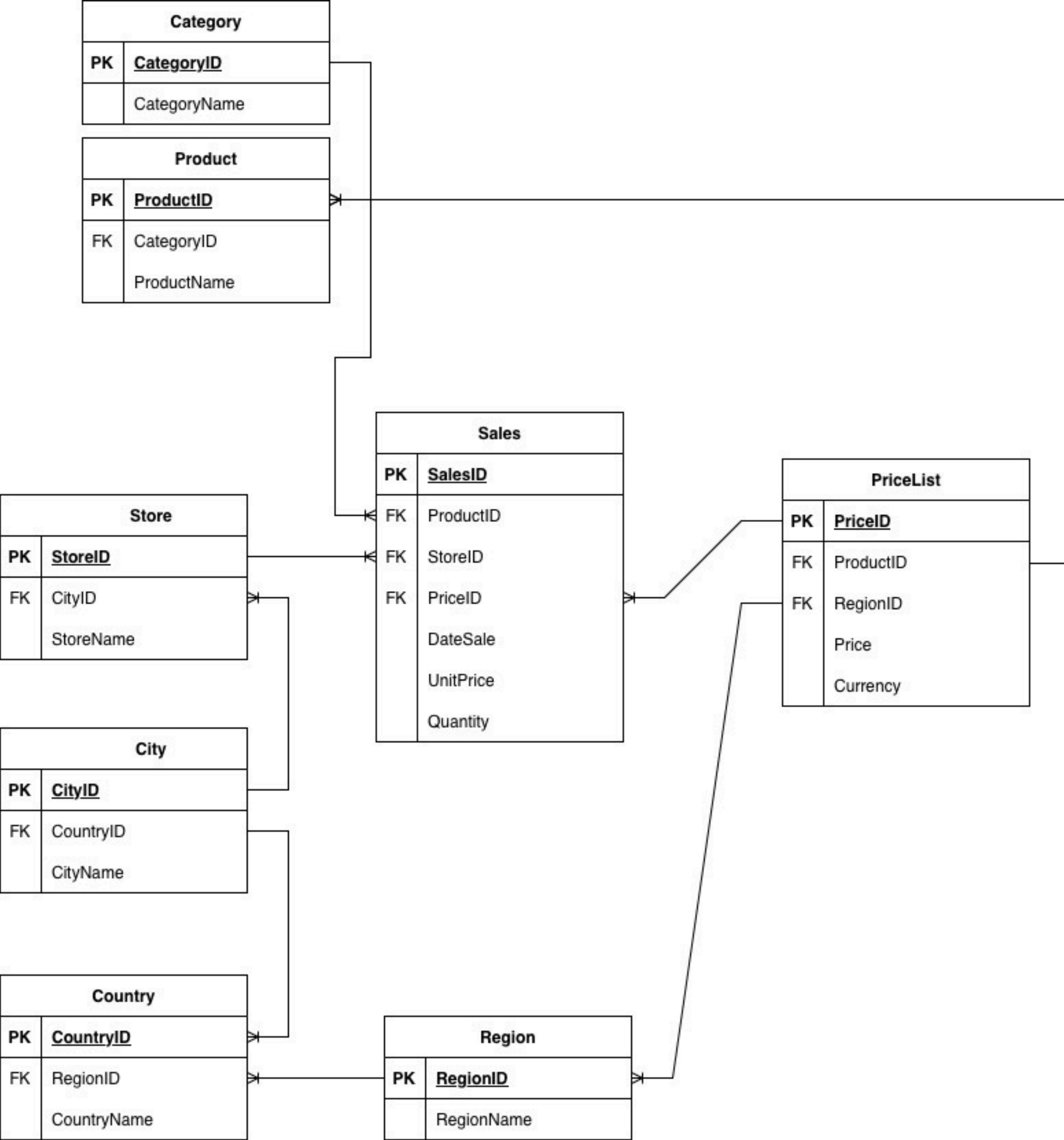
3

Popola le tabelle utilizzando dati a tua discrezione (sono sufficienti pochi record per tabella; riporta le query utilizzate).

4

Dopo aver popolate le tabelle, scrivi delle query utili.

E/R SCHEME



LEGENDA

PK**Chiave Primaria****FK****Chiave Esterna****Entità base**Product, Region,
Category**DimGeografiche**Region, City, Country,
Region**Transazioni**

Sales e PriceList

MODELLO ER:

STEP BY STEP CONSTRUCTION



Nella creazione dello schema E/R ho utilizzato le seguenti tabelle

Entità base

1. **Product**: rappresenta il catalogo prodotti della Toys ed è legata con la tabella Category attraverso una relazione **uno a molti (1:N)**, ad ogni categoria appartengono più prodotti, ogni prodotto è afferente ad una sola categoria. E' legata alla tabella **listprice con una relazione (1:N)**: Un Prodotto (Product) può avere molti prezzi associati nella tabella PriceList. Questo accade perché lo stesso gioco viene venduto in diverse regioni del mondo a prezzi e valute differenti. E' infine legata alla **tabella Sales con una relazione 1:N**, molte vendite possono contenere lo stesso prodotto, ogni prodotto appartiene ad una sola vendita.
2. **Region**: Ogni Regione include più Stati, la tabella region è legata alla **tabella country da una relazione uno a molti (1:N)**, Una Regione (Region) può contenere molti Stati (Country), uno Stato (Country) appartiene a una e una sola Regione. La **tabella region è inoltre legata alla tabella PriceList con una relazione (1:N)**: una Region può essere associata a molte voci nella tabella PriceList, ogni singola riga della PriceList è associata a una sola regione specifica.
3. **Category**: rappresenta le categorie di prodotti, legata alla **tabella product con relazione (1:N)** vista sopra.

Transazioni e prezzi

1. **PriceList**: Entità fondamentale per il contesto internazionale, che ho scelto di inserire perchè aiuta a gestire la combinazione Prodotto-Regione, permettendo di avere prezzi e valute diverse per lo stesso articolo in mercati differenti: **un prodotto non ha un prezzo universale, ma ha un prezzo per ogni regione in cui viene venduto**. Grazie al pricelist inoltre si può **tenere traccia in maniera separata dei prezzi, così che in caso questi cambino ci sarà una tabella specifica in cui riportare i cambiamenti di prezzo**. Pricelist è legata a region e product (vedi spiegaz. sopra) **con due relazioni (1:N)**. Una voce del Listino (PriceList) può essere associata a molte vendite, ogni singola vendita (Sales) fa riferimento a una e una sola riga specifica del listino prezzi.

2. **Sales:** L'entità "**fatto**" che registra ogni **singola vendita**. Collega il prodotto, il negozio e il listino applicato, memorizzando la data, la quantità e il prezzo effettivo di vendita. Nella tabella **sales** oltre che **ListPriceID**, ho inserito il dato **unitprice** in vista di un eventuale cambio di prezzo nel futuro infatti, in questo modo lo storico delle vendite in **unitprice** non verrà intaccato e il DB tutelerà l'integrità dei dati, permettendo anche l'analisi storica dei prezzi e delle loro variazioni nel tempo, oltre che nella dimensione geografica. Come detto, **Sales** è legato alla tabella **store** con una relazione 1:N, ogni vendita è legata a un solo negozio, ogni negozio è legato a più vendite. (per la relazione con **product** e **listprice** vedere sopra).
3. In questo caso non è stato necessario inserire la tabella **salesdetail** in quanto la traccia specificava che per ogni vendita era associato un solo prodotto.

Dimensioni geografiche

Region → Country → City → Store: Ogni livello superiore contiene molti elementi del livello inferiore, ma ogni elemento inferiore appartiene ad una sola tabella padre. Diverse relazioni 1:N

In questo caso ho deciso di creare un quadro di dimensioni geografiche molto esteso che mi permettesse di analizzare non solo **region** e **country**, ma anche **cities** e **stores**, in modo da assicurare una scalabilità del modello stesso in caso ci sia un'estensione in futuro della rete commerciale in altre città.

1. **Country:** ogni Stato contiene più Città, ma una città è legata ad un solo stato (Rel 1:N con **region**)
2. **City:** Ogni città contiene più store, ma uno store è legato ad una sola città (Rel 1:N con **country**)
3. **Store:** Negozi fisici nelle diverse città. (rel 1:N con **city**)

2

CODICE SQL: CREAZIONE DATABASE E TABELLE

```
Create database ToysGroup;      /* Come prima cosa creo il database
                                per la ToysGroup Spa */

Use ToysGroup;

/* procedo qui alla creazione delle tabelle attraverso
istruzioni DDL per le diverse entità protagoniste del DB*/

Create table Category (
CategoryID INT AUTO_INCREMENT PRIMARY KEY,
CategoryName varchar(50)
);

CREATE TABLE Product (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,
    CategoryID INT,
    Foreign key Product(CategoryID) REFERENCES Category(CategoryID)
);

CREATE TABLE Region (
RegionID INT AUTO_INCREMENT PRIMARY KEY,
RegionName VARCHAR(50) NOT NULL
);

CREATE TABLE Country (
CountryID INT AUTO_INCREMENT PRIMARY KEY,
CountryName VARCHAR(50) NOT NULL,
RegionID int,
Foreign key Country(RegionID) REFERENCES Region(RegionID)
);
```

```
CREATE TABLE City (
CityID INT AUTO_INCREMENT PRIMARY KEY,
CityName VARCHAR(50) NOT NULL,
CountryID INT,
Foreign key City(CountryID) REFERENCES Country(CountryID)
);

CREATE TABLE Store (
StoreID INT AUTO_INCREMENT PRIMARY KEY,
StoreName VARCHAR(50) NOT NULL,
CityID INT,
Foreign key Store(CityID) REFERENCES City(CityID)
);

CREATE TABLE PriceList (
PriceListID INT AUTO_INCREMENT PRIMARY KEY,
ProductID INT,
RegionID INT,
Currency VARCHAR(10),
price decimal(10,2),
Foreign key (RegionID) REFERENCES Region(RegionID),
FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);

create table Sales (
SalesID INT AUTO_INCREMENT PRIMARY KEY,
datesale date,
unitprice decimal(10,2),
quantity int,
ProductID int,
StoreID int,
PriceListID int,
Foreign key (ProductID) REFERENCES Product(ProductID),
Foreign key (StoreID) REFERENCES Store(StoreID),
Foreign key (PriceListID) REFERENCES PriceList(PriceListID)
);
```



CODICE SQL: INSERIMENTO DATI

Tabella	Record Inseriti	Descrizione e Logica del Dato
Region	10	Copertura globale suddivisa per macro-aree commerciali (Europa, Asia, Nord America, ecc.).
Country	20	2 Paesi strategici per ogni regione.
City	40	2 Città principali per ogni Paese.
Store	40	1 Punto vendita fisico per ogni città
Category	5	Divisione del catalogo in: Hardware, Videogiochi AAA, Board Games, Puzzle e Accessori.
Product	15	Selezione di prodotti rappresentativi
PriceList	30	Listino prezzi differenziato per Regione e Valuta (EUR, USD, JPY, ecc.)
Sales	50	Transazioni



Nell'inserimento dati ho cercato di assicurare e rispettare i seguenti criteri:

- 1. Integrità Referenziale:** *l'inserimento ha seguito l'ordine gerarchico obbligatorio (prima Region poi Country, in seguito City e infine Store) per rispettare i vincoli di Foreign Key.*
- 2. Realismo del Dataset:** *i prezzi nella PriceList non sono fissi, ma adattati al mercato locale (es. la GameStation costa 549.99€ in Europa e 499.99\$ negli USA).*
- 3. Normalizzazione:** *Lo schema rispetta le forme normali principali, evitando ridondanze di dati.*
- 4. Scalabilità:** *La struttura permette facilmente di aggiungere nuovi punti vendita senza dover modificare la logica delle tabelle esistenti.*
- 5. Distribuzione Temporale:** *Le 50 transazioni di vendita coprono un arco temporale che va da gennaio a marzo 2026.*

TEST DI INTEGRITÀ DEI DATI

La query riportata a sinistra effettua la verifica del criterio di unicità della primary key.

Spiegazione della query:

- Dopo aver selezionato la colonna ID (PRODUCT/CATEGORY ETC..), cioè la Primary key, effettua una conta di quante volte ogni singolo ID appare nella tabella (attraverso count)
- tramite GROUP BY raggruppa tutte le righe che hanno lo stesso codice (stesso id).
- ORDER BY conteggioPK DESC: Ordina i risultati partendo dal numero più alto. Se ci sono duplicati, appariranno in cima alla lista.

Risultati query

- Se conteggioPK è sempre 1 significa che ogni prodotto ha un ID unico e non ci sono inserimenti duplicati. Il database è pulito e coerente.
- Se conteggioPK è > 1: Significa che lo stesso codice prodotto è stato assegnato a due o più righe diverse, quindi non viene rispettato il criterio di unicità della PK.

— Nelle seguente query verrà verificata la presenza della chiave primaria, l'univocità del valore —

```
select productID,
count(productID) as conteggioPK
from product
group by productID
order by conteggiopk DESC;
```

```
select CategoryID,
count(CategoryID) as conteggioPK
from Category
group by CategoryID
order by conteggioPK DESC;
```

```
select CityID,
count(CityID) as conteggioPK
from City
group by cityID
order by conteggiopk DESC;
```

```
select PriceListID,
count(PriceListID) as conteggioPK
from PriceList
group by PriceListID
order by conteggiopk DESC;
```

```
select CountryID,
count(CountryID) as conteggioPK
from Country
group by CountryID
order by conteggiopk DESC;
```

```
select RegionID,
count(RegionID) as conteggioPK
from Region
group by RegionID
order by conteggiopk DESC;
```

```
select SalesID,
count(SalesID) as conteggioPK
from Sales
group by SalesID
order by conteggiopk DESC;
```

```
select StoreID,
count(StoreID) as conteggioPK
from Store
group by StoreID
order by conteggiopk DESC;
```


4

IF E SUBQUERY

La foto riportata a destra contiene:

1. **la prima query** che attraverso una serie di join consente di unire le tabelle necessarie per ottenere i campi richiesti dalla traccia, quindi sales, product, city, store, country e region. Inoltre contiene la funzione booleana IF attraverso la quale si aggiunge un'ulteriore colonna che restituisce TRUE qualora siano passati 180 giorni dalla vendita (differenza calcolata con la funzione DateDiff la quale sottrae alla data odierna la data di vendita), mentre invece restituisce FALSE qualora non siano ancora passati 180 giorni.
2. **la seconda query** utilizza due subquery: la prima nel filtro righe "where" dove dice appunto di prendere i record dove DateSale corrisponde all'anno più grande (Max(year())) di datesale dalla tabella sale. il risultato viene raggruppato per ProductID e i gruppi vengono filtrati in base alla seconda subquery: la somma delle quantità vendute deve essere superiore alla media delle vendite totali raggruppate per prodotto nell'ultimo anno.

```

/* 2) Esporre l'elenco delle transazioni indicando nel result set il codice documento,
la data, il nome del prodotto, la categoria del prodotto, il nome dello stato,
il nome della regione di vendita e un campo booleano valorizzato in base
alla condizione che siano passati più di 180 giorni dalla data vendita o meno (>180 -> True, <= 180 -> False) */
Select
sale.salesID as CodiceProdotto,
sale.Datesale as DataVendita,
Product.ProductName as NomeProdotto,
Category.categoryName as CategoriaProdotto,
Country.CountryName as NomeStato,
Region.RegionName as NomeRegione,
IF(DATEDIFF(CURDATE(), sale.DateSale) > 180, 'True', 'False') AS PiuDi180Giorni
from sales as sale
join product
on Product.ProductID = Sale.ProductID
join category
on product.CategoryID = category.CategoryID
JOIN Store as store
ON sale.StoreID = store.StoreID
JOIN City as city
ON store.CityID = city.CityID
JOIN Country as country
ON city.CountryID = country.CountryID
JOIN Region as region
ON country.RegionID = region.RegionID;

/* Esporre l'elenco dei prodotti che hanno venduto, in totale,
una quantità maggiore della media delle vendite realizzate nell'ultimo anno censito.
(ogni valore della condizione deve risultare da una query e non deve essere inserito a mano).
Nel result set devono comparire solo il codice prodotto e il totale venduto. */
SELECT
ProductID AS Codice_Prodotto,
SUM(Quantity) AS Totale_Venduto
FROM Sales
WHERE YEAR(DateSale) = (SELECT MAX(YEAR(DateSale)) FROM Sales)
GROUP BY ProductID
HAVING SUM(Quantity) > (
/* Subquery per calcolare la media delle vendite
totali per prodotto nell'ultimo anno */
SELECT AVG(TotalePerProdotto)
FROM (
SELECT SUM(Quantity) AS TotalePerProdotto
FROM Sales
WHERE YEAR(DateSale) = (SELECT MAX(YEAR(DateSale)) FROM Sales)
GROUP BY ProductID
) AS MediaVendite
);

```


DOMANDE E QUERY

La foto in questo caso riportano:

1. *nella prima query la funzione di aggregazione somma di quantità vendute da sales e il campo categoria, possibile attraverso una join effettuata tra sales product e category. Il tutto viene raggruppato per categoria, così da riuscire a vedere quale categoria viene maggiormente richiesta dal mercato (quantitativo che viene disposto in maniera decrescente attraverso la funzione ORDER BY).*
2. *Nella seconda foto sono proposte due strade diverse per ottenere lo stesso risultato: i prodotti invenduti. Nella prima query il tutto viene eseguito attraverso una join tra product e sales, il result set è productID productName e si applica il filtro record where cioè mi deve restituire i record in cui salesID = null, cioè non c'è alcuna vendita ID associata al prodotto. Nella seconda query invece si utilizza una subquery, si ricerca quindi il productID NON PRESENTE all'interno dei productID presenti nella tabella sales: qualora non risulti presente infatti, significherebbe che nessuna vendita è associata a quel prodotto.*

```

) /* Rispondere alla seguente domanda:
qual è la categoria di articoli maggiormente
richiesta dal mercato? */

```

SELECT

```

    category.CategoryName AS Categoria,
    SUM(sales.Quantity) AS QuantitaTotaleVenduta

```

FROM Sales **as** sales

JOIN Product **as** product **ON** sales.ProductID = product.ProductID

JOIN Category **as** category **ON** product.CategoryID = category.CategoryID

GROUP BY category.CategoryName

ORDER BY QuantitaTotaleVenduta **DESC**;

```

) /* Rispondere alla seguente domanda: quali sono i prodotti invenduti?
Proponi due approcci risolutivi differenti. */

```

```

/* seleziono tutti i prodotti e cerco quelli che,
dopo il collegamento, non hanno dati di vendita associati. */

```

SELECT

```

prod.ProductID,
prod.ProductName

```

FROM Product **as** prod

LEFT JOIN Sales **as** sal **ON** prod.ProductID = sal.ProductID

WHERE sal.SalesID **IS NULL**;

```

/* in questo modo sto chiedendo di restituirmi
i prodotti il cui ID non è contenuto nell'elenco degli ID venduti */

```

SELECT

```

    ProductID,
    ProductName

```

FROM Product

WHERE ProductID **NOT IN** (

```

    SELECT DISTINCT ProductID

```

```

    FROM Sales

```

```

);

```


GROUP BY

⊖ /* Esporre l'elenco dei soli prodotti venduti
e per ognuno di questi il fatturato totale per anno. */

SELECT

product.ProductName AS NomeProdotto,
YEAR(sales.DateSale) AS AnnoVendita,
SUM(sales.UnitPrice * sales.Quantity) AS FatturatoTotale

FROM Sales as sales

JOIN Product as product

ON sales.ProductID = product.ProductID

GROUP BY product.ProductName, YEAR(sales.DateSale)

ORDER BY AnnoVendita DESC, FatturatoTotale DESC;

⊖ /* Esporre il fatturato totale per stato per anno.
Ordina il risultato per data e per fatturato decrescente. */

SELECT

country.CountryName AS NomeStato,
YEAR(sales.DateSale) AS AnnoVendita,
SUM(sales.UnitPrice * sales.Quantity) AS FatturatoTotale

FROM Sales as sales

JOIN Store as store ON sales.StoreID = store.StoreID

JOIN City as city ON store.CityID = city.CityID

JOIN Country as country ON city.CountryID = country.CountryID

GROUP BY country.CountryName, YEAR(sales.DateSale)

ORDER BY AnnoVendita DESC, FatturatoTotale DESC;

Nella foto riportata a sinistra attraverso l'utilizzo della funzione group by:

- 1. nella prima query consente di, dopo aver unito le tabelle protaoniste sales e product, grazie ad un join attraverso la chiave ProductID, il group by ci consente di ottenere il totale del fatturato raggruppato per nome prodotto e anno*
- 2. nella seconda query invece dopo aver unito attravrso la join le tabelle store, city e country ho ottenuto il fatturato totale stavolta raggruppato per stato e anno*

4

CREATE VIEW

La query riportata a destra effettua la creazione di due view, una sul dettaglio prodotti dove ritroviamo solo codice prodotto, nome prodotto e nome categoria. L'altra una vista geografica che associa ad ogni negozio, la città, lo stato e la regione.

obiettivo delle queries:

- Semplificazione
- Nascondono la complessità del database mostrando solo i nomi reali e parlanti e non più chiavi esterne e ID
- Una volta create, funzionano come "scorciatoie" per qualsiasi altro calcolo

```
/* Creare una vista sui prodotti in modo tale  
da esporre una "versione denormalizzata"  
delle informazioni utili (codice prodotto,  
nome prodotto, nome categoria) */
```

```
CREATE VIEW VistaProdottiDettaglio AS  
SELECT  
    prod.ProductID AS CodiceProdotto,  
    prod.ProductName AS NomeProdotto,  
    cat.CategoryName AS NomeCategoria  
FROM Product as prod  
JOIN Category as cat ON prod.CategoryID = cat.CategoryID;
```

```
/* Creare una vista per le informazioni geografiche */
```

```
CREATE VIEW VistaGeografia AS  
SELECT  
    stor.StoreID AS CodiceNegozio,  
    stor.StoreName AS NomeNegozio,  
    cit.CityName AS Citta,  
    coun.CountryName AS Stato,  
    reg.RegionName AS Regione  
FROM Store as stor  
JOIN City as cit ON stor.CityID = cit.CityID  
JOIN Country as coun ON cit.CountryID = coun.CountryID  
JOIN Region as reg ON coun.RegionID = reg.RegionID;
```


THANK YOU!